CSL7110 — Machine Learning with Big Data

Assignment 4

Clustering and PageRank

Submitted by: Ruby Mythili M

Roll Number: M25DE1006

GitHub Repository:

https://github.com/RubyMythiliM/CSL7110_Assignment4_Clustering_PageRank

Part 1: Clustering

1.1 Objective

The objective of this part is to implement two clustering algorithms on the UCI Spambase dataset and compare their performance. The dataset contains 4601 emails with 58 features each. We implement the Farthest First Traversal algorithm and K-Means++ and evaluate their clustering quality using the k-means objective function.

1.2 Algorithms Implemented

1.2.1 Farthest First Traversal (k-center)

The Farthest First algorithm selects cluster centers in a greedy manner. It begins with the first point in the dataset as the initial center. In each subsequent step, the algorithm selects the point that is farthest from all currently selected centers, measured using squared Euclidean distance. This ensures that centers are spread as far apart as possible across the dataset.

Time Complexity: O(|P| x k) — for each of k centers, all points are scanned once.

1.2.2 K-Means++

K-Means++ improves upon random center initialization. The first center is chosen uniformly at random. Each subsequent center is chosen with probability proportional to its squared distance from the nearest existing center. This probabilistic selection ensures better initial spread of centers, which typically leads to lower objective function values and faster convergence.

Time Complexity: O(|P| x k) — for each of k centers, all points are scanned once.

1.2.3 K-Means Objective Function

The objective function kmeansObj(P, C) measures the quality of a given clustering. For each point in P, it computes the squared distance to its nearest center in C. The average of these squared distances

is returned. A lower value indicates better clustering, meaning points are closer to their assigned centers.

1.3 Experimental Results

The following experiments were conducted with k = 10 and k1 = 50 on the UCI Spambase dataset.

Experiment	Description	Result
Experiment 1	kcenter(P, k=10) — running time	0.9999 seconds
Experiment 2	kmeansPP(P, k=10) — objective value	27619.65
Experiment 3	kcenter(P, k1=50) + kmeansPP(X, k=10) — objective value	65400.66

1.4 Observations

- K-Means++ applied directly on the full dataset gives a much lower objective value (27619.65) compared to the coreset approach (65400.66), indicating tighter and better-quality clusters.
- The Farthest First algorithm is very fast, completing in under 1 second, making it suitable for large-scale datasets where speed is a priority.
- The coreset approach (Experiment 3) uses kcenter to extract k1 representative points, then applies kmeansPP on those points. Increasing k1 beyond 50 would improve the coreset quality.
- The objective function value is a direct measure of cluster compactness — lower values mean points are geometrically closer to their cluster centers.

1.5 Assumptions

- k = 10 and k1 = 50 were used as experimental values.
- The first point in the dataset is used as the initial center for the Farthest First algorithm.
- The random seed is not fixed for K-Means++, so results may vary slightly across different runs.
- Squared Euclidean distance is used throughout for distance computation.

Part 2: Web Search — Inverted Index

2.1 Objective

The objective of this part is to build a search engine from scratch. This involves constructing an Inverted Index from a collection of webpages and using it to answer search queries. The implementation handles stop words, punctuation, and singular-plural normalization as specified in the assignment.

2.2 Key Concepts

2.2.1 Inverted Index

An Inverted Index maps every word to the list of webpages and positions where it appears. This is analogous to the index at the back of a textbook that lists page numbers for important terms. For a collection of webpages C, the inverted index stores, for each word w, all tuples (page, position) where that word occurs.

2.2.2 TF-IDF Scoring

Term Frequency (TF) is the number of occurrences of a word in a webpage divided by the total number of words in that page. Inverse Document Frequency (IDF) is the logarithm of the total number of webpages divided by the number of webpages containing the word. The relevance score is computed as TF x IDF.

2.2.3 Text Preprocessing

Before indexing, all words are converted to lowercase. Stop words such as 'is', 'the', 'of', 'and', 'for' are excluded from the index but are still counted when calculating word positions. Punctuation marks are replaced with spaces. Plural forms such as 'stacks', 'structures', and 'applications' are mapped to their singular equivalents.

2.3 Implementation Details

The search engine processes addPage commands dynamically. A page is added to the index only when an addPage action is explicitly called. Subsequent queries only search within pages that have been added. Query results are returned in alphabetical order.

2.4 Query Results

#	Action	Our Result	Expected	Match
1	queryFindPages: delhi	No webpage contains word delhi	No webpage contains word delhi	Correct
2	queryFindPages: stack	stack_datastructure_wiki	stack_datastructure_wiki	Correct
3	queryFindPages: wikipedia	stack_datastructure_wiki	stack_datastructure_wiki	Correct
4	queryFindPositions: magazines in stack_datastructure_wiki	Webpage does not contain word	Webpage does not contain word	Correct
5	queryFindPages: allain (before addPage)	No webpage contains word allain	No webpage contains word allain	Correct
6	queryFindPages: allain (after addPage)	stack_cprogramming	stack_cprogramming	Correct
7	queryFindPages: C	stack_cprogramming	stack_cprogramming	Correct

#	Action	Our Result	Expected	Match
8	queryFindPages: C++	stack_cprogramming	stack_cprogramming	Correct
9	queryFindPages: jdk	stack_oracle	stack_oracle	Correct
10	queryFindPages: function	stack_cprogramming, stack_datastructure_wiki, stackoverflow	stack_cprogramming, stack_datastructure_wiki, stackoverflow	Correct
11	queryFindPages: magazines	stackmagazine	stackmagazine	Correct

Overall Accuracy: 11 out of 11 queries answered correctly (100%).

2.5 Observations

- All 11 search queries were answered correctly, achieving 100% accuracy.
- The dynamic page-adding feature ensures pages are only searchable after an explicit addPage call. This was verified by the allain query returning different results before and after adding stack_cprogramming.
- Query results are returned in alphabetical order to ensure deterministic and consistent output.
- Stop words are excluded from the index but are still counted when computing word positions, preserving positional accuracy.
- Plural forms such as 'structures' and 'applications' are correctly mapped to their singular forms during indexing.

2.6 Assumptions

- The list of stop words provided in the assignment is treated as exhaustive. No additional stop words were added.
- The list of plural-singular pairs is limited to those specified in the assignment: stacks, structures, applications.
- Query results are sorted alphabetically before being returned.
- Punctuation is removed from words before indexing.

Part 3: PageRank on Apache Spark

3.1 Objective

The objective of this part is to implement the PageRank algorithm using Apache Spark RDDs on a directed graph and identify the most and least important nodes based on their PageRank scores. The algorithm is first validated on a small graph and then run on the full graph with 1000 nodes.

3.2 Algorithm Description

3.2.1 What is PageRank?

PageRank is an algorithm that measures the importance of nodes in a directed graph. A node is considered important if many important nodes point to it. This is a recursive definition — the score of a node depends on the scores of nodes that link to it. The algorithm was originally developed to rank webpages in search engine results.

3.2.2 Iterative Algorithm

The algorithm begins by initializing all node scores equally to 1/n, where n is the total number of nodes. In each iteration, every node distributes its current score equally among all nodes it points to. Each node then collects contributions from all nodes pointing to it. A teleport factor (1 - beta) / n is added to every node to handle rank sinks and ensure convergence. This process is repeated for 40 iterations.

3.2.3 Parameters

Beta (damping factor): 0.8

Teleport probability: 1 - beta = 0.2

Number of iterations: 40

Initial score per node: 1/n

3.3 Results on Small Graph (small.txt)

Rank	Node	PageRank Score
Top 1	Node 53	0.035731
Top 2	Node 14	0.034171
Top 3	Node 40	0.033630
Top 4	Node 1	0.030006
Top 5	Node 27	0.029720
Bottom 5	Node 89	0.003922
Bottom 4	Node 37	0.003808
Bottom 3	Node 81	0.003695
Bottom 2	Node 59	0.003670
Bottom 1	Node 85	0.003410

Validation: Expected top score = 0.036 | Our top score = 0.035731 — Results match the expected value, confirming correctness of the implementation.

3.4 Results on Full Graph (whole.txt)

Rank	Node	PageRank Score
Top 1	Node 263	0.002020
Top 2	Node 537	0.001943
Top 3	Node 965	0.001925
Top 4	Node 243	0.001853
Top 5	Node 285	0.001827
Bottom 5	Node 408	0.000388
Bottom 4	Node 424	0.000355
Bottom 3	Node 62	0.000353
Bottom 2	Node 93	0.000351
Bottom 1	Node 558	0.000329

3.5 Observations

- PageRank scores converge reliably after 40 iterations for both the small and full graphs.
- The top node in the small graph (Node 53, score 0.035731) closely matches the expected value of 0.036, validating the correctness of the implementation.
- In the full graph, Node 263 has the highest PageRank score (0.002020), which is approximately 6 times higher than the lowest-ranked node (Node 558, score 0.000329). This demonstrates a clear importance hierarchy.
- Scores in the full graph are lower than in the small graph because the total probability mass is distributed across 10 times more nodes (1000 vs 100).
- The teleport probability of 0.2 ensures that all nodes receive a minimum baseline score, preventing rank sinks.

3.6 Assumptions

- Multiple directed edges between the same pair of nodes are treated as a single edge, as specified in the assignment.
- Beta = 0.8 and iterations = 40 as specified in the assignment.
- All nodes are initialized equally with score 1/n at the start of the algorithm.
- Nodes with no outgoing edges receive contributions only from the teleport term.

Conclusion

This assignment covered three major topics in Machine Learning with Big Data. All three parts were implemented from scratch, executed successfully, and validated against expected outputs.

Part 1 — Clustering

Two clustering algorithms, Farthest First Traversal and K-Means++, were implemented on the UCI Spambase dataset containing 4601 emails with 58 features. K-Means++ applied directly to the full dataset achieved an objective value of 27619.65, significantly better than the coreset approach value of 65400.66. The Farthest First algorithm completed in under 1 second, demonstrating its efficiency.

Part 2 — Web Search

A complete search engine was built using an Inverted Index. The system correctly answered all 11 search queries with 100% accuracy. The dynamic page-adding feature was successfully validated, ensuring that pages are only indexed and searchable after an explicit addPage call.

Part 3 — PageRank

The PageRank algorithm was implemented using Apache Spark RDDs. The algorithm was first validated on the small graph where the top score of 0.035731 closely matched the expected value of 0.036. On the full graph with 1000 nodes, Node 263 achieved the highest PageRank score of 0.002020, and Node 558 had the lowest score of 0.000329.